CP372 Assignment 2: Reliable Data Transfer over UDP

Vignesh Aila Carson Bolton Hiermona Kidane

Fall 2026

1 Introduction

1.1 Goal of the Assignment

The goal is to send files reliably over UDP. UDP does not guarantee delivery, so we added our own reliability. We built two protocols, Stop-and-Wait and Go-Back-N, in Python. We then dropped packets on purpose (0% to 30%) and compared how fast each protocol transfers files from 10 KB to 100 MB.

1.2 Background

UDP. UDP sends each message as a single packet. It is fast and simple, but it does not check that packets arrive. Packets can be lost, duplicated or arrive out of order, and the sender is never told.

General principle of reliability. To make delivery reliable on top of UDP, we use:

- **Sequence numbers** so the receiver can spot missing or duplicate packets.
- **ACKs** so the receiver can tell the sender what arrived.
- **A timer** so the sender notices when an ACK does not come back.
- **Retransmission** so a lost packet is sent again after a timeout.

Stop-and-Wait. The sender sends one packet and waits for its ACK. If the ACK does not arrive before the timer runs out, it sends the same packet again. Only then does it move to the next packet. It is simple, but only one packet is ever in transit.

Go-Back-N. The sender can send up to N packets (the window) before getting any ACKs. ACKs are cumulative: ACK n means every packet up to n arrived. If the oldest unACKed packet times out, the sender resends it and every packet after it in the window. The receiver only accepts the next packet in order and throws away the rest.

2 Design

Both protocols use the same packet format and the same settings in our tests: 4096 bytes of data per packet and a 0.05 s timeout.

2.1 Stop-and-Wait

Packet format. Each packet has an 11-byte header followed by the data.

Field	Size	Meaning
Type	1 byte	1 = START, 2 = DATA, 3 = ACK, 4 = END
Sequence number	4 bytes	Packet number. START = 0, DATA = 1 to N, END = $N + 1$
ACK number	4 bytes	In an ACK, the packet number being acknowledged
Length	2 bytes	Number of data bytes after the header
Payload	up to 4096 bytes	START: file name and size. DATA: part of the file. ACK/END: empty

Table A. Packet format (same for both protocols)

START tells the receiver to create the file, DATA carries the file, and END tells the receiver to close it.

Timer design. There is one timer for the one packet in transit. After sending, the sender waits up to 0.05 s for the ACK. If the time runs out, it resends the packet and restarts the timer. An old ACK for a different packet does not restart the timer. After 500 timeouts in a row the sender gives up.

ACK handling. The sender only accepts the ACK whose number matches the packet it just sent. The receiver accepts a packet only if it is the one it expects next. It saves the data, sends an ACK and moves on. If a packet arrives that was already saved (because our ACK was late), the receiver does not save it again, but it sends the ACK again.

Window implementation. Stop-and-Wait is a window of size 1. The sender only has to remember the current packet.

Packet loss. The receiver drops each incoming packet with probability equal to the loss rate (`if random.random() < loss: drop`). A dropped packet gets no ACK, so the sender times out and resends it.

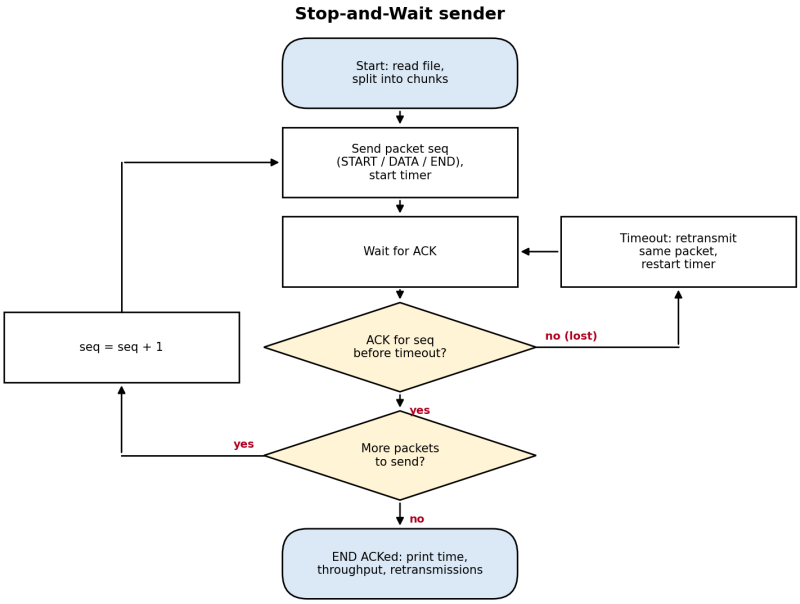


Figure 1: Stop-and-Wait sender: a lost packet causes a timeout, then the same packet is resent

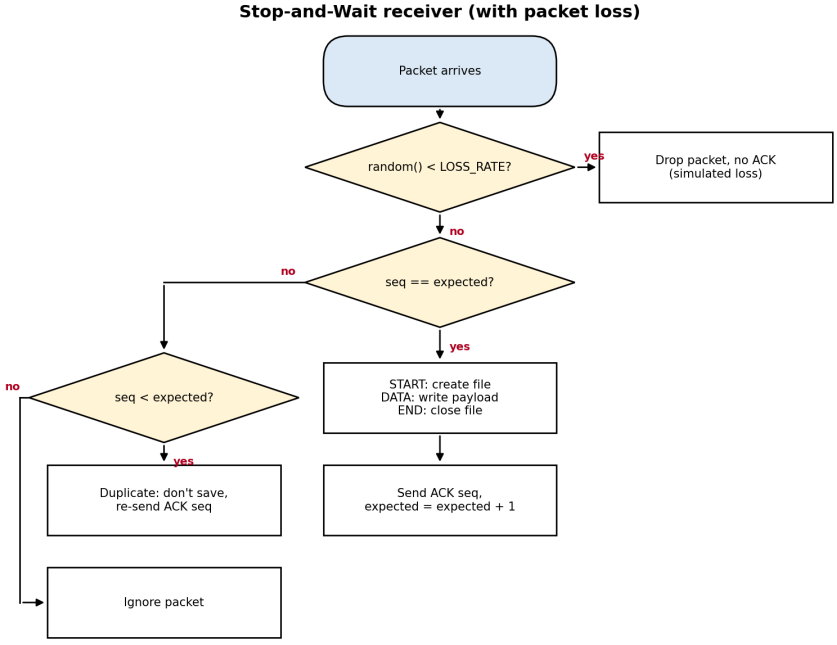


Figure 2: Stop-and-Wait receiver: dropped packets get no ACK; duplicates are ACKed again but not saved

2.2 Go-Back-N

Packet format. Same as Stop-and-Wait (Table A). The only difference is that an ACK number is cumulative: ACK n confirms all DATA packets up to n.

Timer design. START and END are sent one at a time, like Stop-and-Wait, and resent every 0.05 s until ACKed. For DATA packets, the sender records when each packet was sent. The timer belongs to the oldest unACKed packet (`base`). If it has waited 0.05 s, the sender resends every packet from `base` to `nextseqnum` $-$ 1.

ACK handling. When ACK n arrives and is inside the window, the sender moves `base` to $n+1$. Older ACKs are ignored. The receiver accepts only the packet it expects next. For any other packet it discards it and resends the ACK of the last in-order packet. Example: expected = 3, packet 4 arrives, so packet 4 is discarded and ACK 2 is sent. END is only accepted once the whole file has arrived.

Window implementation. The window is the packets from `base` to `base` $+ N - 1$, with $N = 4$. The sender keeps sending while `nextseqnum` $<$ `base` $+ N$. Example: send 1, 2, 3, 4; ACKs 1, 2, 3 arrive, so `base` $= 4$ and packets 5, 6, 7 can be sent.

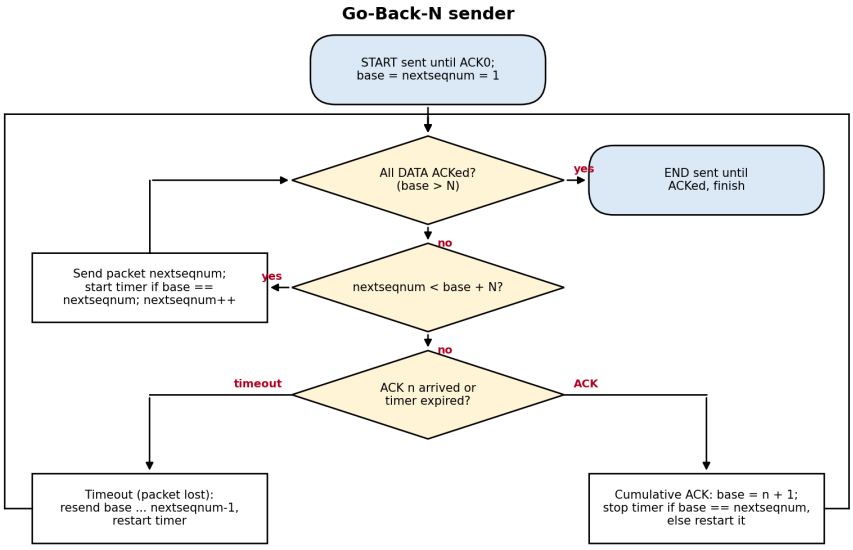


Figure 3: Go-Back-N sender: a timeout on the oldest packet resends the whole window

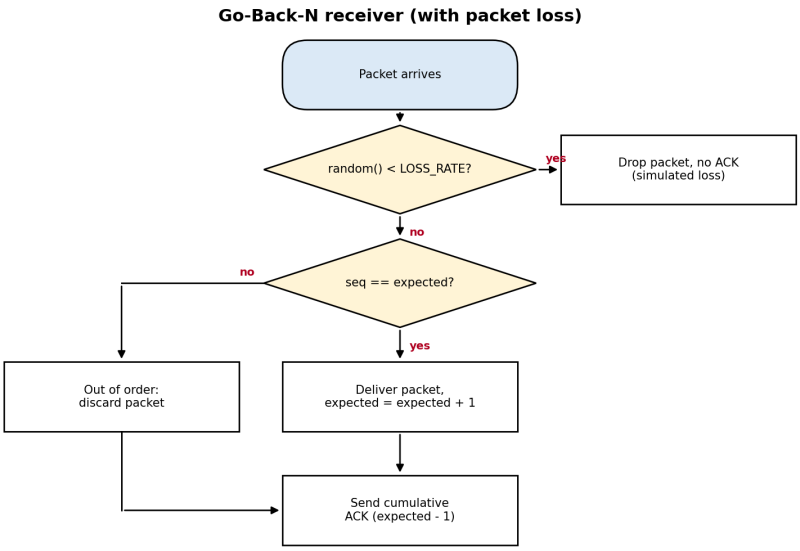


Figure 4: Go-Back-N receiver: after a loss, later packets are discarded until the missing one arrives

3 Experimental Results

All tests ran on one laptop (MacBook Air M4, macOS, Python 3.9) with the sender and receiver on the same machine (127.0.0.1). Each value is the average of 5 tests. Times are in seconds.

Loss Rate	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.0007	0.0015	0.0025	0.0101	0.0160	0.0390	0.0756	0.3483	0.7606
10%	0.0110	0.0837	0.1564	0.7946	1.56	7.40	14.42	73.23	150.9
20%	0.0728	0.2583	0.2709	1.74	3.38	16.58	34.39	163.7	335.5
30%	0.1035	0.2787	0.5973	2.71	5.60	28.26	56.97	285.8	568.1

Table 1. Average Stop-and-Wait transfer time in seconds (average of 5 tests per file size per loss rate)

Loss Rate	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.0008	0.0016	0.0021	0.0068	0.0084	0.0450	0.0861	0.4102	0.9170
10%	0.0118	0.1048	0.1783	0.7410	1.32	7.09	14.54	73.69	146.1
20%	0.0425	0.2176	0.2826	1.58	2.76	16.57	32.38	163.8	327.0
30%	0.1790	0.1972	0.4301	2.80	5.94	29.48	56.35	282.5	561.9

Table 2. Average GBN transfer time in seconds (average of 5 tests per file size per loss rate)

Graphics

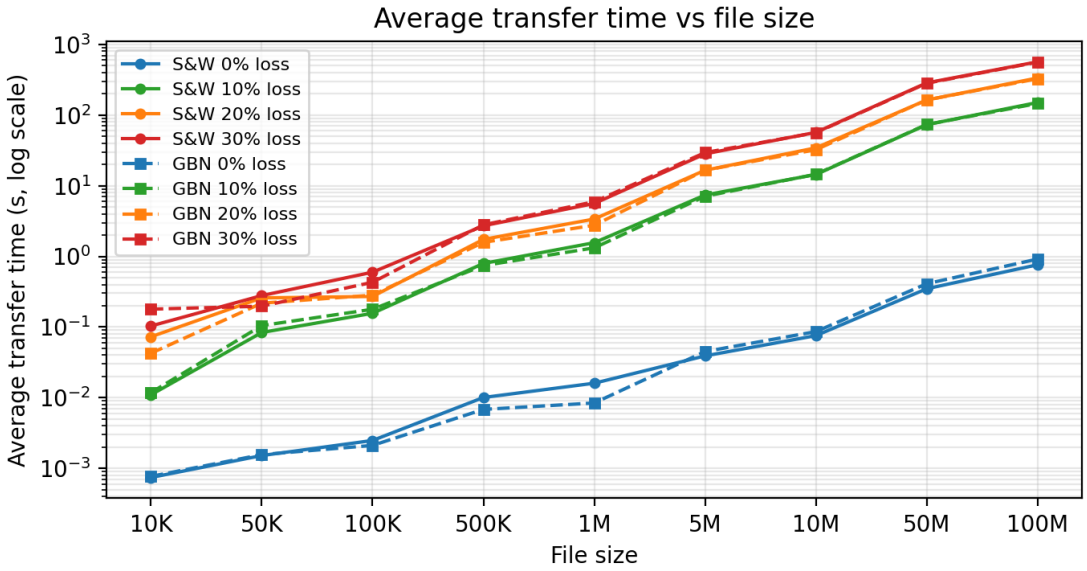


Figure 5: Average transfer time vs. file size (solid = Stop-and-Wait, dashed = GBN)

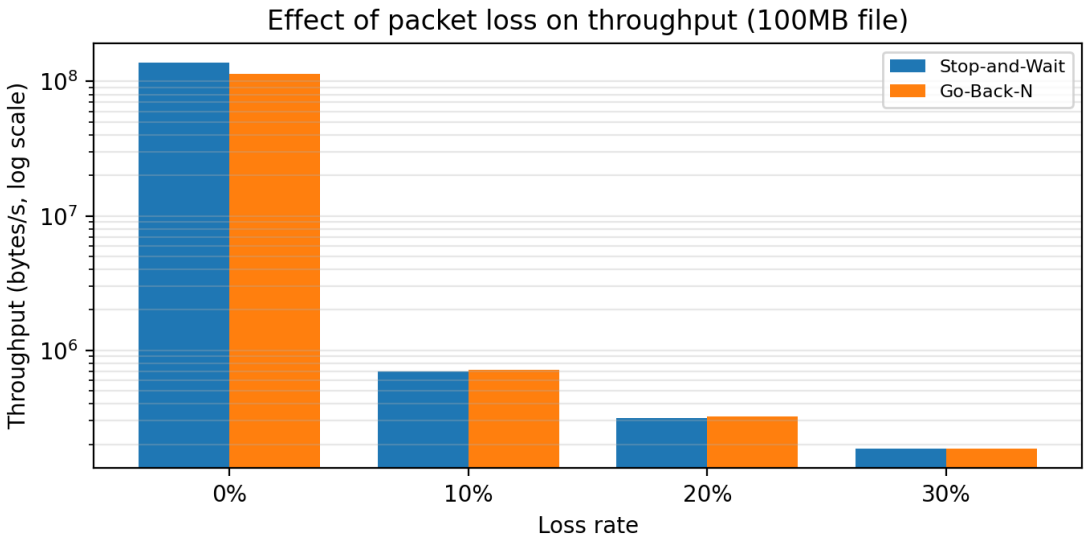


Figure 6: Throughput of the 100 MB file at each loss rate

Loss Rate	S&W resends	GBN resends	S&W useful data	GBN useful data
0%	0	0	100%	100%
10%	2,864	11,304	90%	69%
20%	6,444	25,444	80%	50%
30%	10,962	43,785	70%	37%

Table 3. 100 MB file: number of resent packets, and the share of sent bytes that was new file data

4 Analysis

1. Is GBN always faster than Stop-and-Wait?

No. GBN was faster in 19 of the 36 cases.

- **No loss, 100 KB to 1 MB:** GBN was 1.2 to 1.9 times faster, because it sends 4 packets at a time instead of one.
- **No loss, 5 MB and up:** GBN was 14% to 21% slower. Both programs run on the same machine, so ACKs come back almost instantly and Stop-and-Wait barely waits. The GBN sender also does a bit more work per packet, which adds up over many packets.
- **With loss, 5 MB and up:** both took about the same time (within 6%). Each lost packet costs both protocols one 0.05 s timeout, and these timeouts take up almost all of the time. Sending 4 packets at once does not help, because the packets after a lost one are thrown away and the sender still waits for the timer.
- **Small files with loss:** mixed results. A 10 KB file is only 3 packets, so one extra timeout changes the result.

On a real network with a longer delay, Stop-and-Wait would wait much longer for each ACK, and GBN would be clearly faster.

2. Impact of Increasing Packet Loss

Loss has the biggest effect on time. For the 100 MB file, Stop-and-Wait took 0.76 s with no loss, 151 s at 10% loss and 568 s at 30% loss. GBN was almost the same (0.92 s to 562 s). Every lost packet adds one 0.05 s timeout, so more loss means many more timeouts.

Loss also wastes more data with GBN. Each GBN timeout resends the whole window, so GBN resent about 4 times as many packets (Table 3). At 30% loss only 37% of the bytes GBN sent were new data, compared with 70% for Stop-and-Wait.

3. Trade-offs Between Simplicity and Efficiency

- **Stop-and-Wait** is simple: one packet, one timer, easy to debug. But the sender sits idle while waiting for every ACK.
- **Go-Back-N** keeps more packets in transit, so it uses the connection better. But it needs more state (window, cumulative ACKs, stored packets), and after a loss it resends packets that already arrived.

4. When Stop-and-Wait Is Still Useful

- When the delay is very short, like on the same machine or a local cable, so waiting for an ACK costs little.
- When only a few packets are sent, like short control messages.
- On small devices with little memory that cannot store a window of packets.
- On very lossy links, where GBN would keep resending whole windows.

5 Conclusion

Both protocols delivered every file correctly at 0% to 30% loss. Transfer time grew with file size and grew sharply with loss, because each lost packet costs a full timeout. GBN was only clearly faster for medium files with no loss. With loss, both took about the same time, and GBN resent about 4 times as many packets.

Ways to improve throughput:

- **Shorter, adaptive timeout:** set the timeout from the measured round-trip time instead of a fixed 0.05 s.
- **Fast retransmit:** in GBN, resend as soon as repeated ACKs arrive instead of waiting for the timer.
- **Selective Repeat:** only resend the packets that were lost.
- **Bigger window and bigger packets:** more data in transit and fewer packets per file.